Programação 2

Aula 6 - Memória: memória estática e memória heap, garbage collector, tipos primitivos e referência

LUCAS SILVA FIGUEIREDO
UNIVERSIDADE FEDERAL RURAL DE PERNAMBUCO

Notícias

	Time
(c) C	1.00
(c) Rust	1.04
(c) C++	1.56
(c) Ada	1.85
(v) Java	1.89
(c) Chapel	2.14
(c) Go	2.83
(c) Pascal	3.02
(c) Ocaml	3.09
(v) C#	3.14
(v) Lisp	3.40
(c) Haskell	3.55
(c) Swift	4.20
(c) Fortran	4.20
(v) F#	6.30
(i) JavaScript	6.52
(i) Dart	6.67
(v) Racket	11.27
(i) Hack	26.99
(i) PHP	27.64
(v) Erlang	36.71
(i) Jruby	43.44
(i) TypeScript	46.20
(i) Ruby	59.34
(i) Perl	65.79
(i) Python	71.90
(i) Lua	82.91

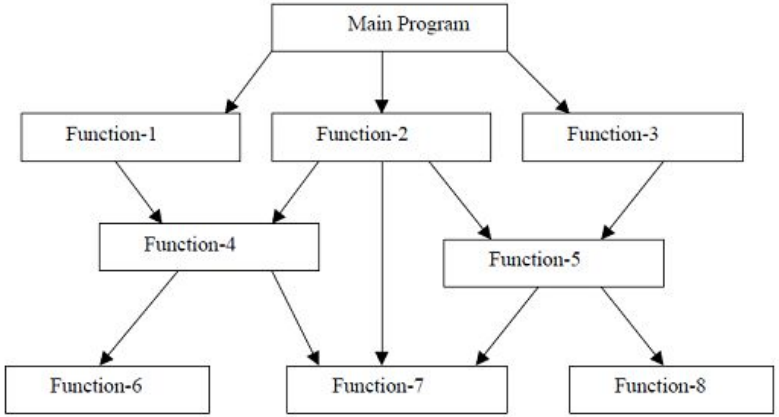
	Mb
(c) Pascal	1.00
(c) Go	1.05
(c) C	1.17
(c) Fortran	1.24
(c) C++	1.34
(c) Ada	1.47
(c) Rust	1.54
(v) Lisp	1.92
(c) Haskell	2.45
(i) PHP	2.57
(c) Swift	2.71
(i) Python	2.80
(c) Ocaml	2.82
(v) C#	2.85
(i) Hack	3.34
(v) Racket	3.52
(i) Ruby	3.97
(c) Chapel	4.00
(v) F#	4.25
(i) JavaScript	4.59
(i) TypeScript	4.69
(v) Java	6.01
(i) Perl	6.62
(i) Lua	6.72
(v) Erlang	7.20
(i) Dart	8.64
(i) Jruby	19.84

	Energy
(c) C	1.00
(c) Rust	1.03
(c) C++	1.34
(c) Ada	1.70
(v) Java	1.98
(c) Pascal	2.14
(c) Chapel	2.18
(v) Lisp	2.27
(c) Ocaml	2.40
(c) Fortran	2.52
(c) Swift	2.79
(c) Haskell	3.10
(v) C#	3.14
(c) Go	3.23
(i) Dart	3.83
(v) F#	4.13
(i) JavaScript	4.45
(v) Racket	7.91
(i) TypeScript	21.50
(i) Hack	24.02
(i) PHP	29.30
(v) Erlang	42.23
(i) Lua	45.98
(i) Jruby	46.54
(i) Ruby	69.91
(i) Python	75.88
(i) Perl	79.58

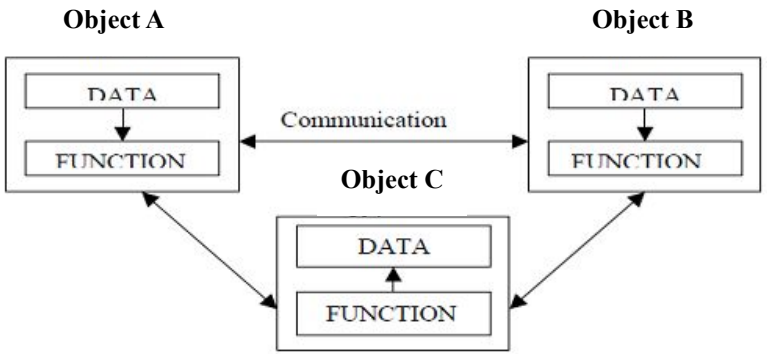
What Are the Greenest Programming Languages? | by Kesk -*- | CodeX | Medium

Recapitulando

Procedural
(ex.: C)



Orientada a Objetos
(ex.: Java)



[Procedure Oriented Programming Vs OOP(Object Oriented Programming) - EXAMRADAR]

Objetos

Objetos

São entidades

- Idealmente com responsabilidades específicas

Guardam dados

Executam métodos

Objetos: exemplos de objetos no nosso código

```
Scanner escaneador = new Scanner(System.in);
```

```
escolha = escaneador.nextLine();

// Em orientação a objetos o próprio objeto avalia seu tamanho, por exemplo
int tamanho = escolha.length();

// Se fosse procedural, seria algo do tipo
int tamanho = funcaoAvaliaTamanho(escolha);
```

Classes

Classe

Imagine a classe como uma fábrica de objetos

É quem “produz” ou “**constrói**” um objeto

Uma fábrica de **pães** produz pães

Uma fábrica de **chocolate** produz chocolate

Uma fábrica de **carros** produz carros

E assim como uma fábrica, a classe pode receber parâmetros para produzir um objeto específico

O tipo de farinha que se usa no pão pode ser um parâmetro

As quantias de açúcar e leite do chocolate...

A cor do carro pode ser um parâmetro

Classe e Objeto

Classe e Objeto

Classe

Objetos produzidos

Parâmetros do solicitante, ex.:
- Cor do carro
- Motor (1.0, 1,6)
- Sedan, hatch

Java (Orientada a Objetos)

Objeto

Entidade, caso real, existência, **instanciação** da classe

Classe

Definição

Quais são as características (atributos, propriedades)?

Quais são as ações (métodos)?

Classe e Objeto sobre a entidade “Cachorro”

Classe

Dados/Atributos

- Raça
- Tamanho
- Cor do Pelo
- Tamanho do Pelo

Ações/Métodos

- Latir
- Morder
- Tremer
- Farejar
- Bagunçar
- Rasgar Sacolas

Objeto

Dados/Atributos

- Raça: Pincher
- Tamanho: 10cm
- Cor do Pelo: marrom
- Tamanho do Pelo: 5mm

Ações/Métodos

- Latir
- Morder
- Tremer
- Farejar
- Bagunçar
- Rasgar Sacolas

Criação, Construtores e **new**

Classes: Criação

Qual cachorro esta classe representa?
Todos e nenhum em específico, é uma "fábrica" de cachorros

O que é o termo "**class**"?
É pra indicar que a gente tá declarando uma classe

O que é o termo "**Cachorro**"?
o nome da classe

O que significa as **chaves** "{ }" ?
Tudo que define essa classe estará entre as chaves, é o escopo da classe

O que é o termo **public**?
...

```
1
2 public class Cachorro {
3
4 }
5
```

Classes: Criação: Atributos: Composição

Objetos de uma classe podem ter **atributos** (*variáveis da instância*) que não são tipos primitivos, mas sim criados a partir de **outras classes**

```java
public class Dog {
    private int age; // Idade
    private float weight; // Peso
    private float height; // Altura
    private String name; // Nome
    private String race; // Raça
    private String personality; // Personalidade
    private Color furColor; // Cor do Pelo

}
```

```java
public class Color {
    private int r;
    private int g;
    private int b;
}
```

Objetos: Criação: Construtores

Alternativamente, podemos criar um **método construtor** que sempre será chamado toda vez que usarmos o comando **new** para a nossa classe.

No construtor podemos indicar todos os parâmetros que consideramos necessários para inicializar o nosso objeto.

Como criar um construtor para a nossa classe?

Objetos: Criação: Construtores

No código...

```java
public class Cor {
    int r;
    int g;
    int b;

    Cor(int r, int g, int b) {
        this.r = r;
        this.g = g;
        this.b = b;
    }
}
```

```java
public class Cachorro {
    String raca;
    float peso;
    float tamanho;
    float tamanhoDoPelo;
    Cor corDoPelo;

    Cachorro(String raca, float peso, float tamanho, float tamanhoDoPelo, Cor corDoPelo)
    {
        this.raca = raca;
        this.peso = peso;
        this.tamanho = tamanho;
        this.tamanhoDoPelo = tamanhoDoPelo;
        this.corDoPelo = corDoPelo;
    }
```

```java
Cor corLabrador = new Cor(170, 140, 110);
Cachorro labrador = new Cachorro("Labrador", 60.f, 0.5f, 0.03f, corLabrador);
```

Conceito de Hoje

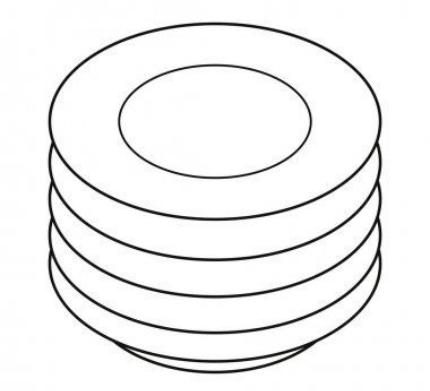
Memória:

Memória stack,
Memória heap,
Garbage collector,
Tipos primitivos e referência

LUCAS SILVA FIGUEIREDO
UNIVERSIDADE FEDERAL RURAL DE PERNAMBUCO

Memória Stack e Heap

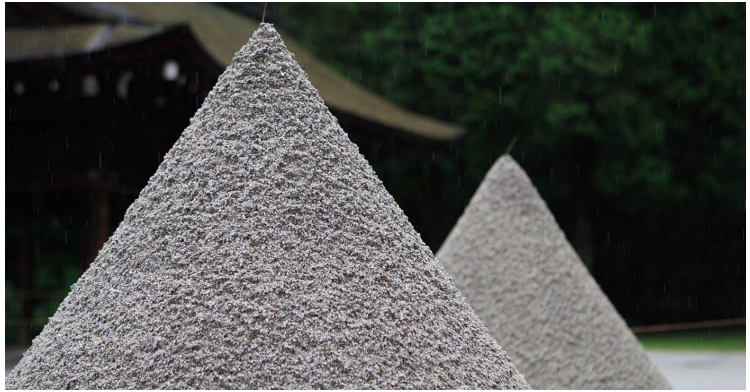
Stack



Na Stack a memória é alocada usando uma lógica parecida com uma pilha de pratos.

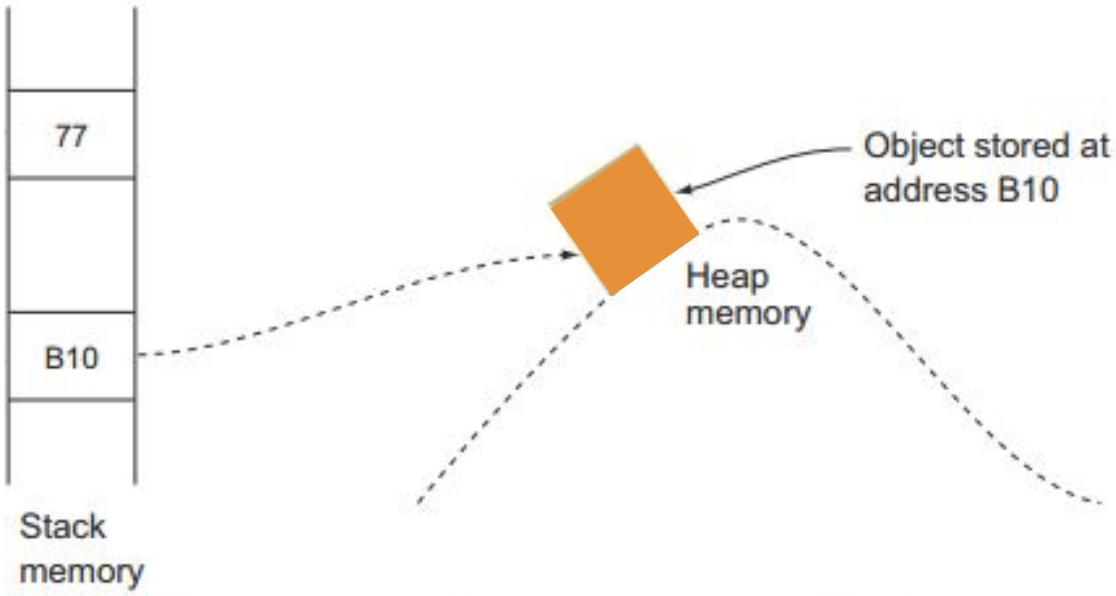
Cada novo espaço de memória ("prato") é adicionado no topo da pilha. Cada espaço só é liberado se estiver no topo da pilha.

Heap



A memória Heap é entendida como mais **dinâmica** e não segue o padrão de pilha para alocação. De forma geral, a memória Heap é maior que a Stack

Memória Stack e Heap



Tipos Primitivos e Referências na Memória

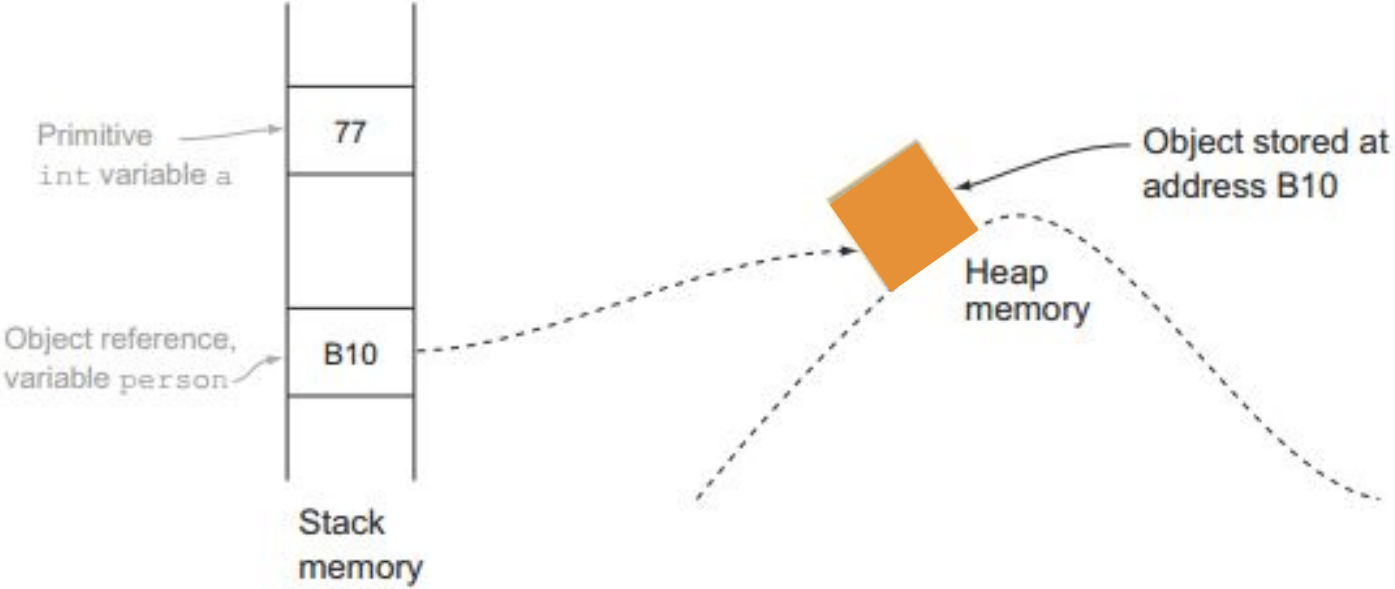
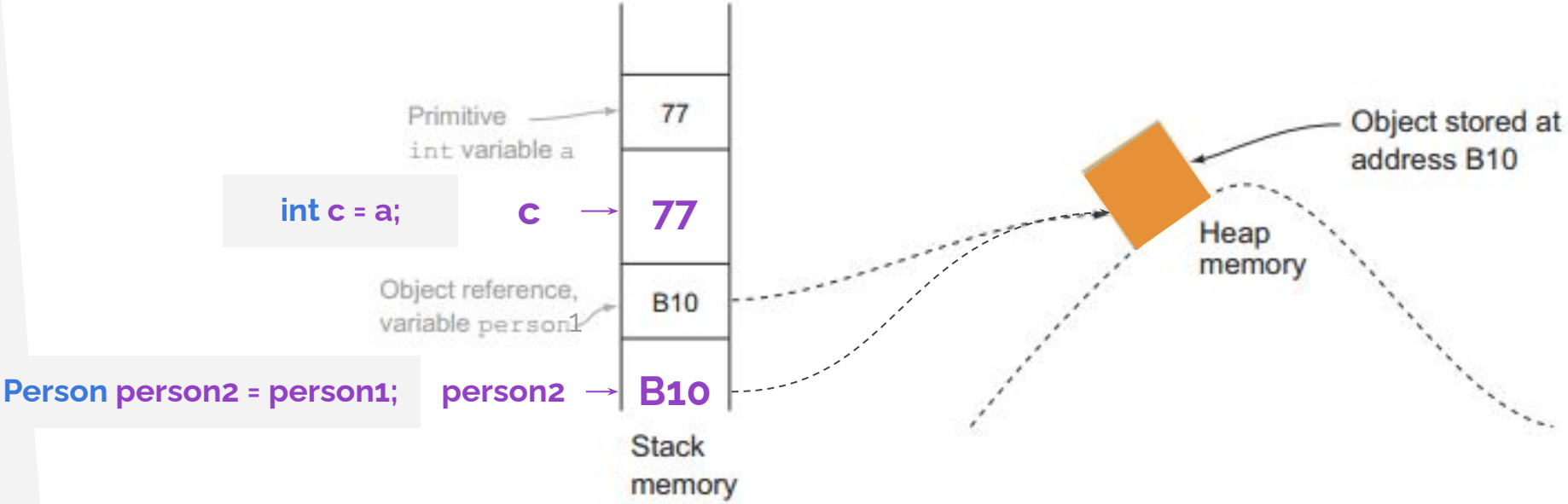


Figure 2.13 Primitive variables store the actual values, whereas object reference variables store the addresses of the objects they refer to.

Tipos Primitivos e Referências na Memória



Tipos Primitivos e Referências na Memória

Tipos Primitivos em Java

- boolean
- byte
- short
- char
- int
- long
- float
- double

Tipos Referência em Java

- Todos os outros
  - Classes
    - String
  - Arrays
  - ...

Tipos Primitivos e Referências na Memória

Alterando o conteúdo de um objeto...

- Diretamente
- Passando como parâmetro em um método
- ...

Tipos Primitivos e Referências na Memória

```
// acréscimo de um ponto para todos
for(float notaAluno : notasAlunos)
{
    notaAluno += 0.5f;
}
```

for que cria uma variável local interna ao for para cada elemento do array

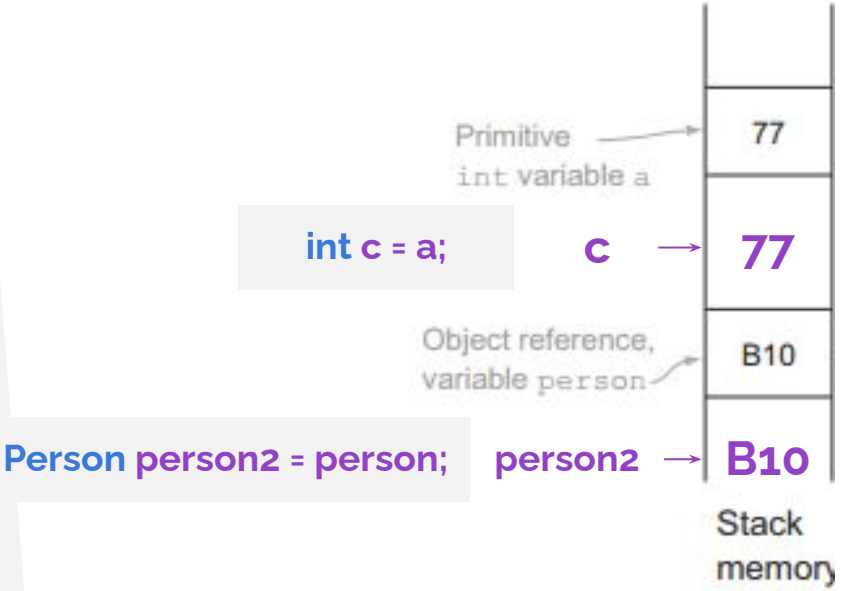
Tipos Primitivos e Referências na Memória

Apesar de mais elegante, essa solução **não funciona** para o propósito de incrementar a nota armazenada no array. Por ser um **tipo primitivo** (float), a variável local é uma **cópia** daquela que existe no array.

```
// acréscimo de um ponto para todos
for(float notaAluno : notasAlunos)
{
    notaAluno += 0.5f;
}
```

for que cria uma variável local interna ao for para cada elemento do array

Tipos Primitivos e Referências na Memória




```
// acréscimo de um ponto para todos
for(float notaAluno : notasAlunos)
{
    notaAluno += 0.5f;
}
```

for que cria uma variável local interna ao for para cada elemento do array

Voltando para o problema, no momento do incremento não estamos mais alterando o **elemento do array que está na Heap**, mas sim uma **cópia do valor que está na Stack**

Tipos Primitivos e Referências na Memória

```
Person[] people = new Person[3];
people[0] = new Person(13);
people[1] = new Person(23);
people[2] = new Person(33);

for(var person : people)
{
    person.age += 1;
}

for(var person : people)
{
    System.out.println(person.age);
}
```

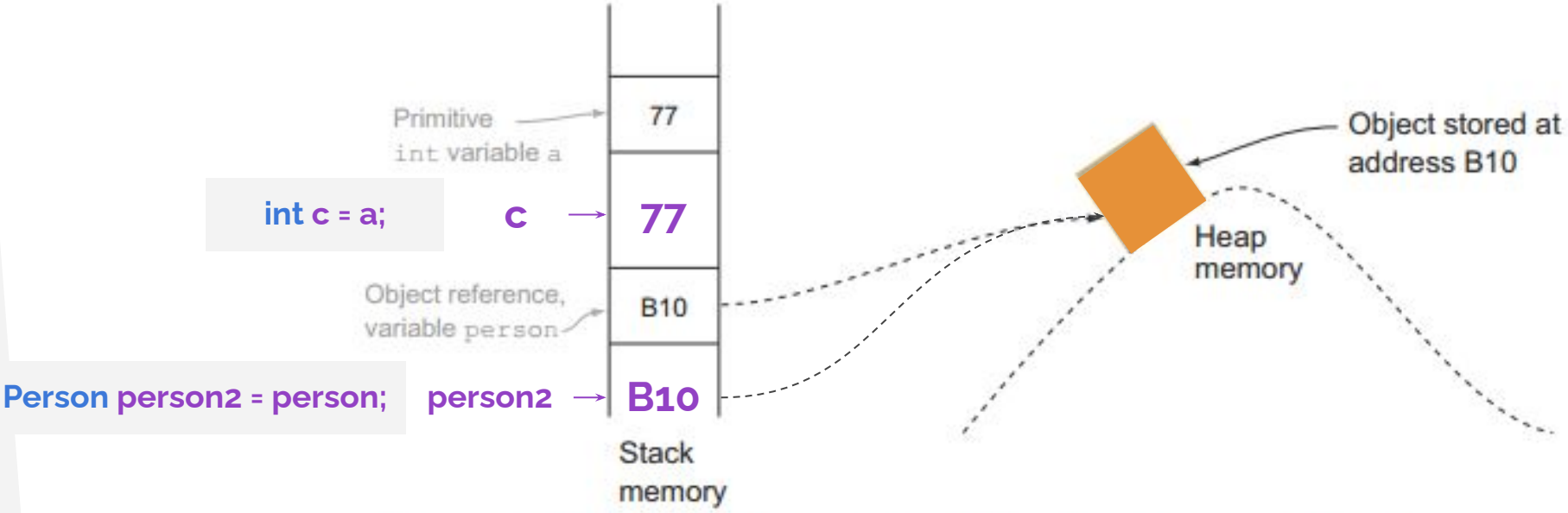
Problems Javadoc Declaration Console

```
<terminated> Main [Java Application] C:\Users\Lucas Figue
14
24
34
```

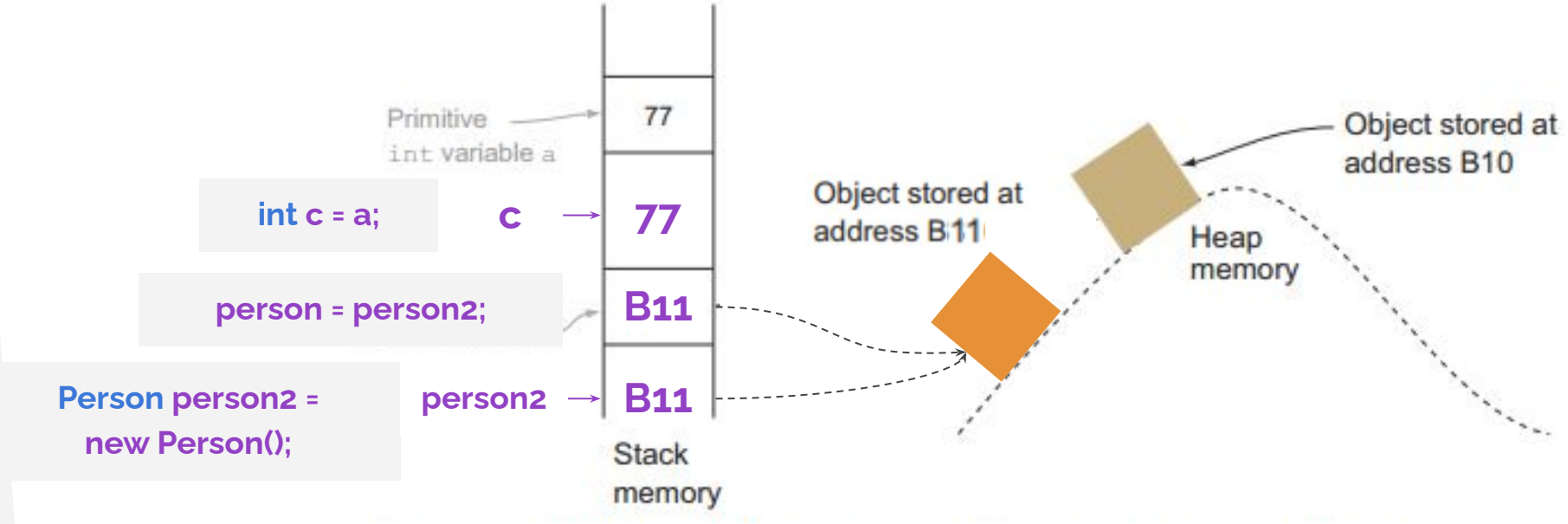
No caso ao lado podemos ver que por usarmos um **tipo referência (Classe Person)**, e acessarmos o conteúdo (**age**) do objeto que **estava na Heap**, o valor de fato **foi modificado** (incremento de 1) como mostrado no resultado do console.

Atenção! Se mudássemos a referência **person = new Person(14);** a alteração não persistiria. Por quê?

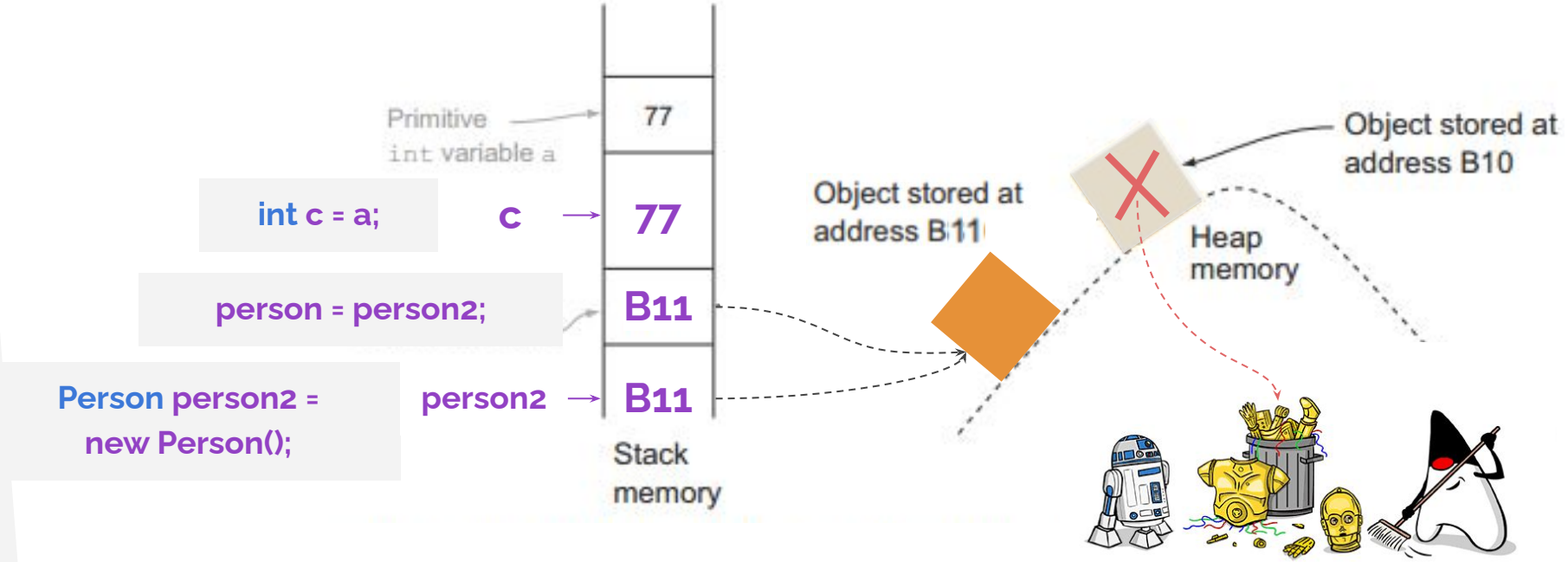
Lixo na Memória Heap e Garbage Collector



Lixo na Memória Heap e Garbage Collector



Lixo na Memória Heap e Garbage Collector



Prática

```java
public static void main(String[] args) throws Exception
{
    int a = 77;
    Person person1 = new Person(name: "Lucy");
    Person person2 = new Person(name: "Derek");
    int b = a + 3;
    person1 = person2;
    person2 = new Person(name: "Carlos");
    Person person3 = new Person(name: "Gleysonn");
    person1 = person3;
    person3.name = "G. Silva";
    System.out.println(person1.name);
    addSenhor(person1);
    System.out.println(person1.name);
}
```

Mini-Projeto 3

Criar uma classe representando um **Capítulo** da história interativa.

Cada **Capítulo** deve ter pelo menos os atributos:
String nome, texto, escolha1, escolha2, **Personagem** personagem, **int** alteracaoEnergia

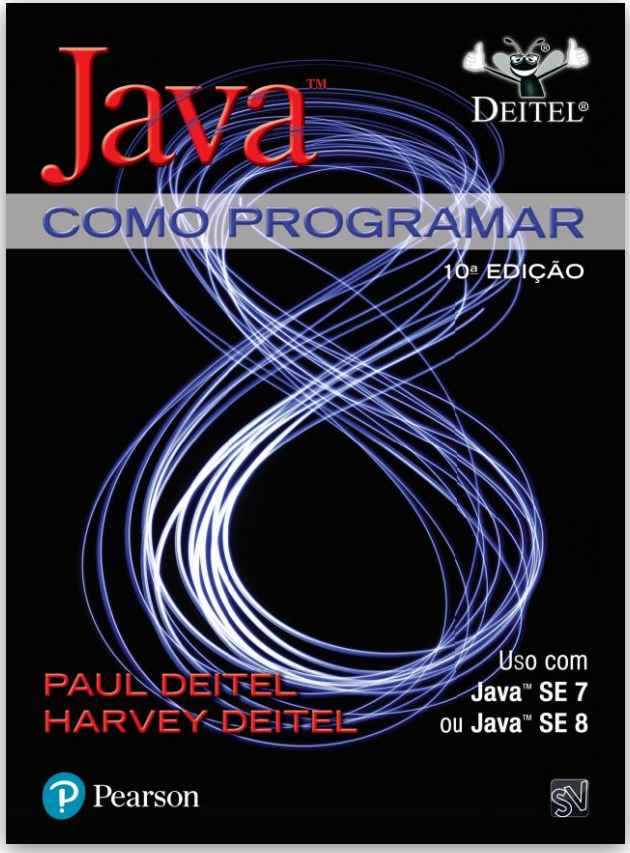
Cada capítulo deve ter um método **void mostrar()** para mostrar o texto e as escolhas no console. Ainda no método mostrar deve ser alterada a energia do personagem de acordo com o valor 'alteracaoEnergia' do capítulo

Cada capítulo deve ter um método **int escolher()** que só termina no momento que o usuário digitar uma escolha válida e que retorna 1 se o texto digitado corresponder à escolha 1, e 2 se corresponder à escolha 2

Todos os textos devem fazer parte de objetos do tipo Capitulo

Referências

Java - Como Programar (10a Edição)

Ferramenta:

Eclipse: eclipse.org/downloads